arXiv:2010.12721v1 [cs.LG] 24 Oct 2020

PEP: Parameter Ensembling by Perturbation

Alireza Mehrtash[1,2], **Purang Abolmaesumi**[1], **Polina Golland**[3], **Tina Kapur**[2], **Demian Wassermann**[4], **William M. Wells III**[2,3]

[1]ECE Department, University of British Columbia (UBC), Vancouver, BC
[2]Department of Radiology, BWH, Harvard Medical School, Boston, MA
[3]CSAIL, MIT, Boston, MA [4]INRIA Saclay, Palaiseau, France

{mehrtash,sw}@bwh.harvard.edu

Abstract

Ensembling is now recognized as an effective approach for increasing the predictive performance and calibration of deep networks. We introduce a new approach, Parameter Ensembling by Perturbation (PEP), that constructs an ensemble of parameter values as random perturbations of the optimal parameter set from training by a Gaussian with a single variance parameter. The variance is chosen to maximize the log-likelihood of the ensemble average ($\mathbb{L}$) on the validation data set. Empirically, and perhaps surprisingly, $\mathbb{L}$ has a well-defined maximum as the variance grows from zero (which corresponds to the baseline model). Conveniently, calibration level of predictions also tends to grow favorably until the peak of $\mathbb{L}$ is reached. In most experiments, PEP provides a small improvement in performance, and, in some cases, a substantial improvement in empirical calibration. We show that this "PEP effect" (the gain in log-likelihood) is related to the mean curvature of the likelihood function and the empirical Fisher information. Experiments on ImageNet pre-trained networks including ResNet, DenseNet, and Inception showed improved calibration and likelihood. We further observed a mild improvement in classification accuracy on these networks. Experiments on classification benchmarks such as MNIST and CIFAR-10 showed improved calibration and likelihood, as well as the relationship between the PEP effect and overfitting; this demonstrates that PEP can be used to probe the level of overfitting that occurred during training. In general, no special training procedure or network architecture is needed, and in the case of pre-trained networks, no additional training is needed.

1 Introduction

Deep neural networks have achieved remarkable success on many classification and regression tasks [28]. In the usual usage, the parameters of a conditional probability model are optimized by maximum likelihood on large amounts of training data [10]. Subsequently the model, in combination with the optimal parameters, is used for inference. Unfortunately, this approach ignores uncertainty in the value of the estimated parameters; as a consequence over-fitting may occur and the results of inference may be overly confident. In some domains, for example medical applications, or automated driving, overconfidence can be dangerous [1].

Probabilistic predictions can be characterized by their level of *calibration*, an empirical measure of consistency with outcomes, and work by Guo et al. shows that modern neural networks (NN) are often poorly calibrated, and that a simple one-parameter *temperature scaling* method can improve their calibration level [12]. Explicitly Bayesian approaches such as *Monte Carlo Dropout* (MCD) [8] have been developed that can improve likelihoods or calibration. MCD approximates a Gaussian process at inference time by running the model several times with active dropout layers. Similar to the

34th Conference on Neural Information Processing Systems (NeurIPS 2020), Vancouver, Canada.

MCD method [8], Teye et al. [45] showed that training NNs with batch normalization (BN) [18] can be used to approximate inference with Bayesian NNs. Directly related to the problem of uncertainty estimation, several works have studied out-of-distribution detection. Hendrycks and Gimpel [14] used softmax prediction probability baseline to effectively predict misclassification and out-of-distribution in test examples. Liang et al. [31] used temperature scaling and input perturbations to enhance the baseline method of Hendrycks and Gimpel [14]. In a recent work, Rohekar et al. [39] proposed a method for confounding training in deep NNs by sharing neural connectivity between generative and discriminative components. They showed that using their BRAINet architecture, which is a hierarchy of deep neural connections, can improve uncertainty estimation. Hendrycks et al. [15] showed that using pre-training can improve uncertainty estimation. Thulasidasan et al. [46] showed that mixed up training can improve calibration and predictive uncertainty of models. Corbière et al. [5] proposed *True Class Probability* as an alternative for classic Maximum Class Probability. They showed that learning the proposed criterion can improve model confidence and failure prediction. Raghu et al. [37] proposed a method for direct uncertainty prediction that can be used for medical second opinions. They showed that deep NNs can be trained to predict uncertainty scores of data instances that have high human reader disagreement.

Ensemble methods [6] are regarded as a straightforward way to increase the performance of base networks and have been used by the top performers in imaging challenges such as ILSVRC [44]. The approach typically prepares an ensemble of parameter values that are used at inference-time to make multiple predictions, using the same base network. Different methods for ensembling have been proposed for improving model performance, such as M-heads [30] and Snapshot Ensembles [16]. Following the success of ensembling methods in improving baseline performance, Lakshminarayanan et al. proposed *Deep Ensembles* in which model averaging is used to estimate predictive uncertainty [26]. By training collections of models with random initialization of parameters and adversarial training, they provided a simple approach to assess uncertainty.

Deep Ensembles and MCD have both been successfully used in several applications for uncertainty estimation and calibration improvement. However, Deep Ensembles requires retraining a model from scratch for several rounds, which is computationally expensive for large datasets and complex models. Moreover, Deep Ensembles cannot be used to calibrate pre-trained networks for which the training data is not available. MCD requires the network architecture to have dropout layers, hence there is a need for network modification if the original architecture does not have dropout layers. In many modern networks, BN removes the need for dropout [18]. It is also challenging or not feasible in some cases to use MCD on out-of-the-box pre-trained networks.

Gaussians are an attractive choice of distribution for going beyond point estimates of network parameters – they are easily sampled to approximate the marginalization that is needed for predictions, and the Laplace approximation can be used to characterize the covariance by using the Hessian of the loss function. Kristiadi et al. [23] support this approach for mitigating the overconfidence of ReLU-based networks. They use a Laplace approximation that is based on the last layer of the network that provides improvements to predictive uncertainty and observe that "a sufficient condition for a calibrated uncertainty on a ReLU network is to be a bit Bayesian." Ritter et al. [38] use a Laplace approach with a layer-wise Kronecker factorization of the covariance that scales only with the square of the size of network layers and obtain improvements similar to dropout. Izmailov et al. [19] describe a stochastic weight averaging Stochastic Weight Averaging (SWA) approach that averages in weight space rather than in model space such as ensembling approaches and approaches that sample distributions on parameters. Averages are calculated over weights observed during training via SGD, leading to wider optima and better generalization in experiments on CIFAR10, CIFAR100 and ImageNet. Building on SWA, Maddox et al. [32] describe Stochastic Weight Averaging-Gaussian (SWAG) that constructs a Gaussian approximation to the posterior on weights. It uses SWA to estimate the first moment on weights combined with a low-rank plus diagonal covariance estimate. They show that SWAG is useful for out of sample detection, calibration and transfer learning.

In this work, we propose Parameter Ensembling by Perturbation (PEP) for deep learning, a simple ensembling approach that uses random perturbations of the optimal parameters from a single training run. PEP is perhaps the simplest possible Laplace approximation - an isotropic Gaussian with one variance parameter, though we set the parameter with simple ML/cross-validation rather than by calculating curvature. Parameter perturbation approaches have been previously used in climate research [33, 2] and they have been used to good effect in variational Bayesian deep learning [21] and to improve adversarial robustness [20].

Unlike MCD which needs dropout at training, PEP can be applied to any pre-trained network without restrictions on the use of dropout layers. Unlike Deep Ensembles, PEP needs only one training run. PEP can provide improved log-likelihood and calibration for classification problems, without the need for specialized or additional training, substantially reducing the computational expense of ensembling. We show empirically that the log-likelihood of the ensemble average ($\mathbb{L}$) on hold-out validation and test data grows initially from that of the baseline model to a well-defined peak as the spread of the parameter ensemble increases. We also show that PEP may be used to probe curvature properties of the likelihood landscape. We conduct experiments on deep and large networks that have been trained on ImageNet (ILSVRC2012) [40] to assess the utility of PEP for improvements on calibration and log-likelihoods. The results show that PEP can be used for probability calibration on pre-trained networks such as DenseNet [17], Inception [44], ResNet [13], and VGG [43]. Improvements in log-likelihood range from small to significant but they are almost always observed in our experiments. To compare PEP with MCD and Deep Ensembles, we ran experiments on classification benchmarks such as MNIST and CIFAR-10 which are small enough for us to re-train and add dropout layers. We carried out an experiment with non-Gaussian perturbations We performed further experiments to study the relationship between over-fitting and the "PEP effect," (the gain in log likelihood over the baseline model) where we observe larger PEP effects for models with higher levels of over-fitting, and finally, we showd that PEP can improve out-of-distribution detection.

To the best of our knowledge, this is the first report of using ensembles of perturbed deep nets as an accessible and computationally inexpensive method for calibration and performance improvement. Our method is potentially most useful when the cost of training from scratch is too high in terms of effort or carbon footprint.

2 Method

In this section, we describe the PEP model and analyze local properties of the resulting PEP effect (the gain in log-likelihood over the comparison baseline model). In summary PEP is formulated in the Bayes' network (hierarchical model) framework; it constructs ensembles by Gaussian perturbations of the optimal parameters from training. The single variance parameter is chosen to maximize the likelihood of ensemble average predictions on validation data, which, empirically, has a well-defined maximum. PEP can be applied to any pre-trained network; only one standard training run is needed, and no special training or network architecture is needed.

2.1 Baseline Model

We begin with a standard discriminative model, e.g., a classifier that predicts a distribution on y_i given an observation x_i,

$$p(y_i; x_i, \theta) \ . \tag{1}$$

Training is conventionally accomplished by maximum likelihood,

$$\theta^* \doteq \operatorname*{argmax}_{\theta} \mathcal{L}(\theta) \qquad \text{where the log-likelihood is:} \qquad \mathcal{L}(\theta) \doteq \sum_i \ln L_i(\theta) \ , \tag{2}$$

and $L_i(\theta) \doteq p(y_i; x_i, \theta)$ are the individual likelihoods. Subsequent predictions are made with the model using θ^*.

2.2 Hierarchical Model

Empirically, different optimal values of θ are obtained on different data sets; we aim to model this variability with a very simple parametric model – an isotropic normal distribution with mean and scalar variance parameters,

$$p(\theta; \bar{\theta}, \sigma) \doteq N(\theta; \bar{\theta}, \sigma^2 I) \ . \tag{3}$$

The product of Eqs. 1 and 3 specifies a joint distribution on y_i and θ; from this we can obtain model predictions by marginalizing over θ, which leads to

$$p(y_i; x_i, \bar{\theta}, \sigma) = \mathbb{E}_{\theta \sim N(\bar{\theta}, \sigma^2 I)} \left[p(y_i; x_i, \theta) \right] \ . \tag{4}$$